

=====

CAPSTONE PROJECT

NOTEBOOK 45: FULL-161 RARE-TAIL PROTOTYPE RERANKER

=====

Purpose:

This notebook tries to improve Stage-2
rare-tail performance

by replacing the simple anchor-only router
with a prototype /

nearest-neighbor reranker for rare-tail
labels that have at

least some training support.

The goal is to:

1. Load the frozen Stage-1 benchmark
model
2. Build training banks for rare-tail labels

3. Compare multiple Stage-2 rare-tail strategies:

- baseline anchor-prior
- prototype centroid similarity
- nearest-neighbor similarity
- hybrid prior + nearest-neighbor

4. Tune the best Stage-2 strategy on targeted validation tracks

5. Evaluate it on targeted rare-tail test tracks

6. Save report-ready comparison tables

=====

In [1]:

```
# =====  
# 1. IMPORT LIBRARIES  
# =====  
  
import os  
import warnings  
from pathlib import Path  
  
import joblib  
import librosa  
import numpy as np  
import pandas as pd  
import tensorflow as tf  
  
warnings.filterwarnings("ignore", category=FutureWarning)
```

```
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

Seed set to: 42
TensorFlow version: 2.20.0

In [2]:

```
# =====
# 2. LOAD FROZEN ARTIFACTS
# =====

structured_model =
joblib.load("../models/final_structured_multilabel_candidate150_best_model.joblib")
structured_scaler =
joblib.load("../models/final_structured_multilabel_candidate150_scaler.joblib")
audio_model =
tf.keras.models.load_model("../models/audio_multilabel_candidate150_expanded_final.
keras")

candidate_label_cols = np.load(
    "../data/processed/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

stage1_config = {}
with
open("../data/processed/hybrid_multilabel_candidate150_expanded_best_config.txt",
"r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            stage1_config[k.strip()] = float(v.strip())

STAGE1_STRUCTURED_WEIGHT = stage1_config["structured_weight"]
STAGE1_AUDIO_WEIGHT = stage1_config["audio_weight"]
STAGE1_THRESHOLD = stage1_config["threshold"]

rare_tail_router_df =
pd.read_csv("../data/processed/full161_rare_tail_routing_table.csv")
genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")

features_reference = pd.read_csv(
    "../data/raw/metadata/features.csv",
    header=[0, 1, 2],
    index_col=0
)
```

```

print("Structured model loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Stage-1 config:", stage1_config)
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Full master shape:", full_master_df.shape)
print("Reference features shape:", features_reference.shape)

```

Structured model loaded.

Audio model loaded.

Candidate labels: 150

Stage-1 config: {'structured_weight': 0.1, 'audio_weight': 0.9, 'threshold': 0.2}

Rare-tail router shape: (13, 26)

Full master shape: (81574, 170)

Reference features shape: (106574, 518)

In [3]:

```

# =====
# 3. PREPARE LOOKUPS AND FEATURE TABLE
# =====

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)
genre_name_map = dict(zip(genre_inventory_df["genre_id"],
                           genre_inventory_df["genre_name"]))

candidate_label_ids = [int(col.replace("genre_", "")) for col in
                        candidate_label_cols]
candidate_id_to_index = {int(col.replace("genre_", "")): i for i, col in
                          enumerate(candidate_label_cols)}

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

fallback_rare_ids = fallback_router_df["rare_tail_genre_id"].astype(int).tolist()
fallback_rare_cols = [f"genre_{gid}" for gid in fallback_rare_ids]

full_master_indexed = full_master_df.set_index("track_id", drop=False)

# Flatten feature columns to match structured branch format
features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df.index = reference_feature_df.index.astype(int)
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)

```

```
reference_feature_columns = list(reference_feature_df.columns)

print("Fallback rare-tail labels:", len(fallback_rare_ids))
print("Structured reference feature columns:", len(reference_feature_columns))
display(fallback_router_df)
```

Fallback rare-tail labels: 10
Structured reference feature columns: 518

	rare_tail_genre_id	rare_tail_genre_name	parent_id	parent_name	root_genre_id	root_genre_
0	176	Pacific	2	International	2	Interna
1	1060	Tango	46	Latin America	2	Interna
2	465	Musical Theater	20	Spoken	20	Sp
3	189	Talk Radio	65	Radio	20	Sp
4	1032	Turkish	102	Middle East	2	Interna
5	374	Banter	20	Spoken	20	Sp
6	173	N. Indian Traditional	86	Indian	2	Interna
7	493	Western Swing	651	Country & Western	9	Co
8	377	Deep Funk	19	Funk	14	Sou
9	808	Salsa	46	Latin America	2	Interna

10 rows × 26 columns



In [4]:

```

# =====
# 4. BUILD TARGETED VALIDATION / TEST COHORTS
# =====

work_df = full_master_df.copy()
work_df["true_fallback_rare_tail_count"] = work_df[fallback_rare_cols].sum(axis=1)

targeted_val_df = work_df[
    (work_df["split"] == "validation") &
    (work_df["true_fallback_rare_tail_count"] > 0)
].copy().reset_index(drop=True)

targeted_test_df = work_df[
    (work_df["split"] == "test") &
    (work_df["true_fallback_rare_tail_count"] > 0)
].copy().reset_index(drop=True)

rare_train_df = work_df[
    (work_df["split"] == "training") &
    (work_df["true_fallback_rare_tail_count"] > 0)
].copy().reset_index(drop=True)

print("Targeted validation cohort shape:", targeted_val_df.shape)
print("Targeted test cohort shape:", targeted_test_df.shape)
print("Rare-tail training cohort shape:", rare_train_df.shape)

display(targeted_val_df[["track_id", "title", "audio_path",
    "true_fallback_rare_tail_count"]].head(20))
display(targeted_test_df[["track_id", "title", "audio_path",
    "true_fallback_rare_tail_count"]].head(20))
display(rare_train_df[["track_id", "title", "audio_path",
    "true_fallback_rare_tail_count"]].head(20))

```

Targeted validation cohort shape: (14, 171)

Targeted test cohort shape: (35, 171)

Rare-tail training cohort shape: (60, 171)

	track_id	title	audio_path	true_fallback_rare_tail_count
0	19777	Custer: "If I Were An Indian..."	../data/raw/audio/fma_large\019\019777.mp3	1
1	19778	Custer's Ghost To Sitting Bull	../data/raw/audio/fma_large\019\019778.mp3	1
2	19779	Sitting Bull: "Do You Know Who I Am?"	../data/raw/audio/fma_large\019\019779.mp3	1
3	19780	Sun Dance / Battle Of The Greasy Grass River	../data/raw/audio/fma_large\019\019780.mp3	1
4	35855	Klingon Rock	../data/raw/audio/fma_large\035\035855.mp3	1
5	35856	Coconut Wireless	../data/raw/audio/fma_large\035\035856.mp3	1
6	38741	Rose Room	../data/raw/audio/fma_large\038\038741.mp3	1
7	47843	MAMA - 110515	../data/raw/audio/fma_large\047\047843.mp3	1
8	61461	D'Bronx Tanz	../data/raw/audio/fma_large\061\061461.mp3	1
9	61462	Een Laatste Liedje	../data/raw/audio/fma_large\061\061462.mp3	1
10	61464	Kolomeika	../data/raw/audio/fma_large\061\061464.mp3	1
11	61465	Ojos Negros	../data/raw/audio/fma_large\061\061465.mp3	1
12	61467	Planta Baja	../data/raw/audio/fma_large\061\061467.mp3	1
13	61506	Kölderren Polka	../data/raw/audio/fma_large\061\061506.mp3	1



	track_id	title	audio_path	true_fallback_rare_tail_cou
0	16930	Open Session	../data/raw/audio/fma_large\016\016930.mp3	
1	30180	Conic Sections	../data/raw/audio/fma_large\030\030180.mp3	
2	30181	Elliptically	../data/raw/audio/fma_large\030\030181.mp3	
3	30182	Hyperboli	../data/raw/audio/fma_large\030\030182.mp3	
4	30183	Directrix	../data/raw/audio/fma_large\030\030183.mp3	
5	30184	So It Goes	../data/raw/audio/fma_large\030\030184.mp3	
6	30185	Quitting Time	../data/raw/audio/fma_large\030\030185.mp3	
7	30186	Roundabout	../data/raw/audio/fma_large\030\030186.mp3	
8	30187	Inscribed Pythagorus	../data/raw/audio/fma_large\030\030187.mp3	
9	30188	The Electricity Song	../data/raw/audio/fma_large\030\030188.mp3	
10	30189	Unit Circle Trigonometry	../data/raw/audio/fma_large\030\030189.mp3	
11	35853	Untitled	../data/raw/audio/fma_large\035\035853.mp3	
12	35854	Untitled	../data/raw/audio/fma_large\035\035854.mp3	
13	35857	Fanfare for the King of Tonga	../data/raw/audio/fma_large\035\035857.mp3	
14	35858	The King's Swim Fins	../data/raw/audio/fma_large\035\035858.mp3	
15	51013	The City	../data/raw/audio/fma_large\051\051013.mp3	
16	51014	Black-Bar Trombone	../data/raw/audio/fma_large\051\051014.mp3	
17	99351	Ille de Roman Olsun	../data/raw/audio/fma_large\099\099351.mp3	
18	99352	Osman Aga	../data/raw/audio/fma_large\099\099352.mp3	
19	99353	Zeytinyagli	../data/raw/audio/fma_large\099\099353.mp3	



	track_id	title	audio_path	true_fallback_rare_tail_cou
0	21268	Seed	../data/raw/audio/fma_large\021\021268.mp3	
1	21269	Bedtime	../data/raw/audio/fma_large\021\021269.mp3	
2	21270	A Leaf's Lament	../data/raw/audio/fma_large\021\021270.mp3	
3	21271	A Bird Named Bob	../data/raw/audio/fma_large\021\021271.mp3	
4	21272	Crow Berry	../data/raw/audio/fma_large\021\021272.mp3	
5	21273	Moon II	../data/raw/audio/fma_large\021\021273.mp3	
6	21274	The Whirlwinds Grandmother	../data/raw/audio/fma_large\021\021274.mp3	
7	23153	Embellir (with Les Gauchers Quintet)	../data/raw/audio/fma_large\023\023153.mp3	
8	27962	(c) Warrior Woman That I Am	../data/raw/audio/fma_large\027\027962.mp3	
9	36515	Intro	../data/raw/audio/fma_large\036\036515.mp3	
10	36517	[banter]	../data/raw/audio/fma_large\036\036517.mp3	
11	36519	[banter]	../data/raw/audio/fma_large\036\036519.mp3	
12	36522	[station ID]	../data/raw/audio/fma_large\036\036522.mp3	
13	36524	[banter]	../data/raw/audio/fma_large\036\036524.mp3	
14	44953	Guiro para Elegua	../data/raw/audio/fma_large\044\044953.mp3	
15	54646	clouds form	../data/raw/audio/fma_large\054\054646.mp3	
16	54647	a feather for frog's first wing	../data/raw/audio/fma_large\054\054647.mp3	
17	54648	old blood	../data/raw/audio/fma_large\054\054648.mp3	
18	54650	happy finite	../data/raw/audio/fma_large\054\054650.mp3	
19	54651	lambs quarters	../data/raw/audio/fma_large\054\054651.mp3	



In [5]:

=====
5. ALIGN AND SCALE STRUCTURED FEATURES

```
# =====

def align_feature_bank(track_ids, feature_df, scaler):
    available_ids = [int(tid) for tid in track_ids if int(tid) in feature_df.index]
    missing_ids = sorted(list(set([int(tid) for tid in track_ids]) -
set(available_ids)))

    X = feature_df.loc[available_ids, :].copy()
    X = X.fillna(X.mean(axis=0))
    X_scaled = scaler.transform(X).astype(np.float32)

    X_scaled_df = pd.DataFrame(X_scaled, index=available_ids, columns=X.columns)
    return X_scaled_df, missing_ids

train_feature_bank_df, missing_train_ids = align_feature_bank(
    rare_train_df["track_id"].tolist(),
    reference_feature_df,
    structured_scaler
)

val_feature_bank_df, missing_val_ids = align_feature_bank(
    targeted_val_df["track_id"].tolist(),
    reference_feature_df,
    structured_scaler
)

test_feature_bank_df, missing_test_ids = align_feature_bank(
    targeted_test_df["track_id"].tolist(),
    reference_feature_df,
    structured_scaler
)

print("Train feature bank shape:", train_feature_bank_df.shape)
print("Validation feature bank shape:", val_feature_bank_df.shape)
print("Test feature bank shape:", test_feature_bank_df.shape)
print("Missing train IDs:", len(missing_train_ids))
print("Missing val IDs:", len(missing_val_ids))
print("Missing test IDs:", len(missing_test_ids))
```

```
Train feature bank shape: (60, 518)
Validation feature bank shape: (14, 518)
Test feature bank shape: (35, 518)
Missing train IDs: 0
Missing val IDs: 0
Missing test IDs: 0
```

In [6]:

```
# =====
# 6. AUDIO / INFERENCE SETTINGS
# =====

SR = 22050
DURATION = 15
N_MELS = 64
N_FFT = 2048
```

```

HOP_LENGTH = 1024
MAX_FRAMES = int(np.ceil((DURATION * SR) / HOP_LENGTH)) + 1

print("SR:", SR)
print("DURATION:", DURATION)
print("N_MELS:", N_MELS)
print("MAX_FRAMES:", MAX_FRAMES)

```

```

SR: 22050
DURATION: 15
N_MELS: 64
MAX_FRAMES: 324

```

In [7]:

```

# =====
# 7. HELPER FUNCTIONS
# =====

def load_audio(file_path, sr=SR, duration=DURATION):
    y, sr_loaded = librosa.load(file_path, sr=sr, mono=True, duration=duration)
    if y is None or len(y) == 0:
        raise ValueError(f"Could not load usable audio from: {file_path}")
    return y, sr_loaded

def build_mel_input(y, sr=SR, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH,
max_frames=MAX_FRAMES):
    mel = librosa.feature.melspectrogram(
        y=y,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels
    )
    mel_db = librosa.power_to_db(mel, ref=np.max)
    mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

    if mel_db.shape[1] < max_frames:
        pad_width = max_frames - mel_db.shape[1]
        mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
    else:
        mel_db = mel_db[:, :max_frames]

    return mel_db.astype(np.float32)[None, :, :, None]

def scores_to_pseudopros(score_matrix):
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))

def get_structured_scores(model, X_scaled):
    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_scaled)
    elif hasattr(model, "decision_function"):
        scores = model.decision_function(X_scaled)
    else:
        raise ValueError("Structured model supports neither predict_proba nor

```

```

decision_function.")
    return np.asarray(scores)

def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    return (w_structured * structured_probs) + (w_audio * audio_probs)

def decode_stage1(prob_matrix, threshold):
    return (prob_matrix >= threshold).astype(np.uint8)

def cosine_sim(vec_a, vec_b):
    a = np.asarray(vec_a, dtype=np.float64)
    b = np.asarray(vec_b, dtype=np.float64)

    a_norm = np.linalg.norm(a)
    b_norm = np.linalg.norm(b)

    if a_norm == 0 or b_norm == 0:
        return 0.0

    sim = float(np.dot(a, b) / (a_norm * b_norm))
    sim = max(-1.0, min(1.0, sim))
    return sim

def cosine_sim_scaled_01(vec_a, vec_b):
    return (cosine_sim(vec_a, vec_b) + 1.0) / 2.0

def get_true_labels(track_id):
    row = full_master_indexed.loc[track_id]

    true_candidate_ids = []
    for col in candidate_label_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    true_rare_ids = []
    for col in fallback_rare_cols:
        if col in row.index and int(row[col]) == 1:
            true_rare_ids.append(int(col.replace("genre_", "")))

    return (
        true_candidate_ids,
        [genre_name_map.get(gid, str(gid)) for gid in true_candidate_ids],
        true_rare_ids,
        [genre_name_map.get(gid, str(gid)) for gid in true_rare_ids],
    )

```

In [8]:

```

# =====
# 8. BUILD RARE-TAIL TRAINING BANKS / PROTOTYPES
# =====

max_prior = fallback_router_df["p_rare_given_anchor"].fillna(0.0).max()
if max_prior <= 0:
    max_prior = 1.0

```

```

rare_tail_bank = {}

for _, row in fallback_router_df.iterrows():
    gid = int(row["rare_tail_genre_id"])
    anchor_id = int(row["anchor_candidate_id"])
    col = f"genre_{gid}"

    train_ids = rare_train_df.loc[rare_train_df[col] == 1,
    "track_id"].astype(int).tolist()
    train_ids = [tid for tid in train_ids if tid in train_feature_bank_df.index]

    if len(train_ids) == 0:
        continue

    member_matrix = train_feature_bank_df.loc[train_ids].values.astype(np.float32)
    centroid = member_matrix.mean(axis=0).astype(np.float32)

    prior_norm = float(row["p_rare_given_anchor"]) / max_prior if not
pd.isna(row["p_rare_given_anchor"]) else 0.0

    rare_tail_bank[gid] = {
        "rare_tail_genre_id": gid,
        "rare_tail_genre_name": row["rare_tail_genre_name"],
        "anchor_candidate_id": anchor_id,
        "anchor_candidate_name": row["anchor_candidate_name"],
        "training_track_ids": train_ids,
        "member_matrix": member_matrix,
        "centroid": centroid,
        "prior_norm": prior_norm,
        "training_count": len(train_ids),
    }

print("Rare-tail bank size:", len(rare_tail_bank))
display(pd.DataFrame([
    {
        "rare_tail_genre_id": v["rare_tail_genre_id"],
        "rare_tail_genre_name": v["rare_tail_genre_name"],
        "anchor_candidate_id": v["anchor_candidate_id"],
        "anchor_candidate_name": v["anchor_candidate_name"],
        "training_count": v["training_count"],
        "prior_norm": v["prior_norm"]
    }
    for v in rare_tail_bank.values()
]).sort_values(["training_count", "rare_tail_genre_name"], ascending=[False,
True]).reset_index(drop=True))

```

Rare-tail bank size: 10

	rare_tail_genre_id	rare_tail_genre_name	anchor_candidate_id	anchor_candidate_name	traini
0	176	Pacific	2	International	
1	189	Talk Radio	65	Radio	
2	1032	Turkish	102	Middle East	
3	374	Banter	20	Spoken	
4	1060	Tango	46	Latin America	
5	465	Musical Theater	20	Spoken	
6	173	N. Indian Traditional	86	Indian	
7	377	Deep Funk	19	Funk	
8	808	Salsa	46	Latin America	
9	493	Western Swing	651	Country & Western	

In [9]:

```

# =====
# 9. CACHE STAGE-1 OUTPUTS FOR TARGETED VALIDATION / TEST TRACKS
# =====

def build_stage1_cache(cohort_df, feature_bank_df, cohort_name="cohort"):
    cache = {}
    problems = []

    for _, row in cohort_df.iterrows():
        track_id = int(row["track_id"])
        audio_path = row["audio_path"]

        if track_id not in feature_bank_df.index:
            problems.append({
                "track_id": track_id,
                "audio_path": audio_path,
                "error": "Missing structured feature row"
            })
            continue

        try:
            y, sr_loaded = load_audio(audio_path, sr=SR, duration=DURATION)
            X_audio_input = build_mel_input(
                y,
                sr=sr_loaded,
                n_mels=N_MELS,
                n_fft=N_FFT,
                hop_length=HOP_LENGTH,
                max_frames=MAX_FRAMES
            )

            structured_vec =
feature_bank_df.loc[[track_id]].values.astype(np.float32)

```

```

        structured_scores = get_structured_scores(structured_model,
structured_vec)
        structured_probs = scores_to_pseudoprobs(structured_scores)

        audio_probs = audio_model.predict(X_audio_input, verbose=0)

        stage1_probs = fuse_probabilities(
            structured_probs,
            audio_probs,
            STAGE1_STRUCTURED_WEIGHT,
            STAGE1_AUDIO_WEIGHT
        )

        stage1_pred = decode_stage1(stage1_probs, STAGE1_THRESHOLD)

        predicted_candidate_ids = [
            int(col.replace("genre_", ""))
            for j, col in enumerate(candidate_label_cols)
            if int(stage1_pred[0, j]) == 1
        ]
        predicted_candidate_names = [genre_name_map.get(gid, str(gid)) for gid
in predicted_candidate_ids]

        true_candidate_ids, true_candidate_names, true_rare_ids,
true_rare_names = get_true_labels(track_id)

        cache[track_id] = {
            "track_id": track_id,
            "title": row.get("title", None),
            "audio_path": audio_path,
            "query_vec": structured_vec[0],
            "stage1_probs": stage1_probs[0],
            "predicted_candidate_ids": predicted_candidate_ids,
            "predicted_candidate_names": predicted_candidate_names,
            "true_candidate_ids": true_candidate_ids,
            "true_candidate_names": true_candidate_names,
            "true_rare_ids": true_rare_ids,
            "true_rare_names": true_rare_names,
        }

    except Exception as e:
        problems.append({
            "track_id": track_id,
            "audio_path": audio_path,
            "error": str(e)
        })

    print(f"{cohort_name} cache size:", len(cache))
    print(f"{cohort_name} problem files:", len(problems))
    return cache, problems

val_cache, val_problem_files = build_stage1_cache(targeted_val_df,
val_feature_bank_df, cohort_name="validation")
test_cache, test_problem_files = build_stage1_cache(targeted_test_df,
test_feature_bank_df, cohort_name="test")

```

```
validation cache size: 14
validation problem files: 0
test cache size: 35
test problem files: 0
```

In [10]:

```
# =====
# 10. DEFINE STAGE-2 RERANKING STRATEGIES
# =====

def build_candidate_stage2_scores(stage1_probs, query_vec, trigger_threshold,
strategy_name):
    candidates = []

    for gid, bank_item in rare_tail_bank.items():
        anchor_id = bank_item["anchor_candidate_id"]
        anchor_idx = candidate_id_to_index[anchor_id]
        anchor_prob = float(stage1_probs[anchor_idx])

        if anchor_prob < trigger_threshold:
            continue

        prior_norm = float(bank_item["prior_norm"])
        centroid_sim = cosine_sim_scaled_01(query_vec, bank_item["centroid"])

        nn_sims = [cosine_sim_scaled_01(query_vec, member_vec) for member_vec in
bank_item["member_matrix"]]
        nn_sim = max(nn_sims) if len(nn_sims) > 0 else 0.0

        if strategy_name == "baseline_prior":
            stage2_score = anchor_prob * prior_norm

        elif strategy_name == "prototype_centroid":
            stage2_score = anchor_prob * centroid_sim

        elif strategy_name == "nearest_neighbor":
            stage2_score = anchor_prob * nn_sim

        elif strategy_name == "hybrid_nn_prior_25":
            stage2_score = anchor_prob * (0.25 * nn_sim + 0.75 * prior_norm)

        elif strategy_name == "hybrid_nn_prior_50":
            stage2_score = anchor_prob * (0.50 * nn_sim + 0.50 * prior_norm)

        elif strategy_name == "hybrid_nn_prior_75":
            stage2_score = anchor_prob * (0.75 * nn_sim + 0.25 * prior_norm)

        else:
            raise ValueError(f"Unknown strategy: {strategy_name}")

        candidates.append({
            "rare_tail_genre_id": gid,
            "rare_tail_genre_name": bank_item["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
```



```

        "anchor_candidate_name": bank_item["anchor_candidate_name"],
        "anchor_prob": anchor_prob,
        "prior_norm": prior_norm,
        "centroid_sim": centroid_sim,
        "nn_sim": nn_sim,
        "stage2_score": stage2_score,
        "strategy_name": strategy_name
    })

    if len(candidates) == 0:
        return pd.DataFrame(columns=[
            "rare_tail_genre_id", "rare_tail_genre_name",
            "anchor_candidate_id", "anchor_candidate_name",
            "anchor_prob", "prior_norm", "centroid_sim", "nn_sim",
            "stage2_score", "strategy_name"
        ])

    candidates_df = pd.DataFrame(candidates).sort_values(
        ["stage2_score", "anchor_prob", "nn_sim", "centroid_sim"],
        ascending=False
    ).reset_index(drop=True)

    return candidates_df

def evaluate_strategy_on_cache(cache, strategy_name, trigger_threshold):
    rows = []

    for track_id, item in cache.items():
        candidate_scores_df = build_candidate_stage2_scores(
            item["stage1_probs"],
            item["query_vec"],
            trigger_threshold=trigger_threshold,
            strategy_name=strategy_name
        )

        if len(candidate_scores_df) > 0:
            top_row = candidate_scores_df.iloc[0]
            suggested_ids = [int(top_row["rare_tail_genre_id"])]
            suggested_names = [top_row["rare_tail_genre_name"]]
        else:
            suggested_ids = []
            suggested_names = []

        hit_any = any(gid in suggested_ids for gid in item["true_rare_ids"]) if
len(item["true_rare_ids"]) > 0 else False
        hit_top1 = hit_any

        rows.append({
            "track_id": track_id,
            "title": item["title"],
            "true_rare_tail_label_names": item["true_rare_names"],
            "suggested_rare_tail_names": suggested_names,
            "suggestion_count": len(suggested_ids),
            "rare_tail_hit_any": hit_any,
            "rare_tail_hit_exact_top1": hit_top1
        })

```

```

eval_df = pd.DataFrame(rows)

n_tracks = len(eval_df)
tracks_with_suggestion = int((eval_df["suggestion_count"] > 0).sum()) if
n_tracks > 0 else 0
hits_any = int(eval_df["rare_tail_hit_any"].sum()) if n_tracks > 0 else 0
hits_top1 = int(eval_df["rare_tail_hit_exact_top1"].sum()) if n_tracks > 0 else
0

summary = {
    "Strategy": strategy_name,
    "Anchor Trigger Threshold": trigger_threshold,
    "Eligible Tracks": n_tracks,
    "Tracks With Suggestion": tracks_with_suggestion,
    "Suggestion Coverage": (tracks_with_suggestion / n_tracks) if n_tracks > 0
else np.nan,
    "Rare-Tail Hit-Any Count": hits_any,
    "Rare-Tail Hit-Any Rate": (hits_any / n_tracks) if n_tracks > 0 else
np.nan,
    "Rare-Tail Top-1 Hit Count": hits_top1,
    "Rare-Tail Top-1 Hit Rate": (hits_top1 / n_tracks) if n_tracks > 0 else
np.nan,
}

return eval_df, summary

```

In [11]:

```

# =====
# 11. VALIDATION SEARCH ACROSS STAGE-2 STRATEGIES
# =====

strategy_list = [
    "baseline_prior",
    "prototype_centroid",
    "nearest_neighbor",
    "hybrid_nn_prior_25",
    "hybrid_nn_prior_50",
    "hybrid_nn_prior_75",
]

trigger_grid = [0.05, 0.10, 0.15, 0.20]

validation_search_rows = []
validation_eval_store = {}

for strategy_name in strategy_list:
    for trigger_threshold in trigger_grid:
        eval_df, summary = evaluate_strategy_on_cache(
            val_cache,
            strategy_name=strategy_name,
            trigger_threshold=trigger_threshold
        )
        validation_search_rows.append(summary)
        validation_eval_store[(strategy_name, trigger_threshold)] = eval_df

```

```
validation_search_df = pd.DataFrame(validation_search_rows).sort_values(  
    [  
        "Rare-Tail Hit-Any Rate",  
        "Rare-Tail Top-1 Hit Rate",  
        "Suggestion Coverage"  
    ],  
    ascending=False  
) .reset_index(drop=True)  
  
print("Validation Stage-2 strategy search results:")  
display(validation_search_df)
```

Validation Stage-2 strategy search results:

	Strategy	Anchor Trigger Threshold	Eligible Tracks	Tracks With Suggestion	Suggestion Coverage	Rare- Tail Hit- Any Count	Rare- Tail Hit- Any Rate	Rare- Tail Top-1 Hit Count
0	baseline_prior	0.05	14	14	1.000000	2	0.142857	2
1	baseline_prior	0.10	14	13	0.928571	2	0.142857	2
2	baseline_prior	0.15	14	11	0.785714	2	0.142857	2
3	prototype_centroid	0.05	14	14	1.000000	1	0.071429	1
4	nearest_neighbor	0.05	14	14	1.000000	1	0.071429	1
5	hybrid_nn_prior_25	0.05	14	14	1.000000	1	0.071429	1
6	hybrid_nn_prior_50	0.05	14	14	1.000000	1	0.071429	1
7	hybrid_nn_prior_75	0.05	14	14	1.000000	1	0.071429	1
8	prototype_centroid	0.10	14	13	0.928571	1	0.071429	1
9	nearest_neighbor	0.10	14	13	0.928571	1	0.071429	1
10	hybrid_nn_prior_25	0.10	14	13	0.928571	1	0.071429	1
11	hybrid_nn_prior_50	0.10	14	13	0.928571	1	0.071429	1
12	hybrid_nn_prior_75	0.10	14	13	0.928571	1	0.071429	1
13	prototype_centroid	0.15	14	11	0.785714	1	0.071429	1
14	nearest_neighbor	0.15	14	11	0.785714	1	0.071429	1
15	hybrid_nn_prior_25	0.15	14	11	0.785714	1	0.071429	1
16	hybrid_nn_prior_50	0.15	14	11	0.785714	1	0.071429	1
17	hybrid_nn_prior_75	0.15	14	11	0.785714	1	0.071429	1
18	baseline_prior	0.20	14	8	0.571429	0	0.000000	0
19	prototype_centroid	0.20	14	8	0.571429	0	0.000000	0
20	nearest_neighbor	0.20	14	8	0.571429	0	0.000000	0
21	hybrid_nn_prior_25	0.20	14	8	0.571429	0	0.000000	0
22	hybrid_nn_prior_50	0.20	14	8	0.571429	0	0.000000	0
23	hybrid_nn_prior_75	0.20	14	8	0.571429	0	0.000000	0

In [12]:

```
# =====  
# 12. SELECT BEST STAGE-2 STRATEGY  
# =====
```

```

best_row = validation_search_df.iloc[0].to_dict()
BEST_STRATEGY = best_row["Strategy"]
BEST_TRIGGER_THRESHOLD = float(best_row["Anchor Trigger Threshold"])

print("Best Stage-2 reranking strategy:")
print("Strategy:", BEST_STRATEGY)
print("Anchor trigger threshold:", BEST_TRIGGER_THRESHOLD)
print("Best validation row:")
print(best_row)

```

Best Stage-2 reranking strategy:

Strategy: baseline_prior

Anchor trigger threshold: 0.05

Best validation row:

```
{'Strategy': 'baseline_prior', 'Anchor Trigger Threshold': 0.05, 'Eligible Tracks': 14, 'Tracks With Suggestion': 14, 'Suggestion Coverage': 1.0, 'Rare-Tail Hit-Any Count': 2, 'Rare-Tail Hit-Any Rate': 0.14285714285714285, 'Rare-Tail Top-1 Hit Count': 2, 'Rare-Tail Top-1 Hit Rate': 0.14285714285714285}
```

In [13]:

```

# =====
# 13. EVALUATE BASELINE VS BEST RERANKER ON TARGETED TEST
# =====

baseline_test_df, baseline_test_summary = evaluate_strategy_on_cache(
    test_cache,
    strategy_name="baseline_prior",
    trigger_threshold=0.10
)

best_test_df, best_test_summary = evaluate_strategy_on_cache(
    test_cache,
    strategy_name=BEST_STRATEGY,
    trigger_threshold=BEST_TRIGGER_THRESHOLD
)

comparison_df = pd.DataFrame([baseline_test_summary, best_test_summary])

print("Targeted test comparison:")
display(comparison_df)

```

Targeted test comparison:

	Strategy	Anchor Trigger Threshold	Eligible Tracks	Tracks With Suggestion	Suggestion Coverage	Rare-Tail Hit-Any Count	Rare-Tail Hit-Any Rate	Rare-Tail Top-1 Hit Count	Rare-Tail Top-1 Hit Rate
0	baseline_prior	0.10	35	33	0.942857	10	0.285714	10	0.285714
1	baseline_prior	0.05	35	34	0.971429	11	0.314286	11	0.314286

In [14]:

```

# =====
# 14. BUILD FINAL TARGETED TEST RESULTS TABLE
# =====

final_rows = []

for track_id, item in test_cache.items():
    baseline_scores_df = build_candidate_stage2_scores(
        item["stage1_probs"],
        item["query_vec"],
        trigger_threshold=0.10,
        strategy_name="baseline_prior"
    )

    best_scores_df = build_candidate_stage2_scores(
        item["stage1_probs"],
        item["query_vec"],
        trigger_threshold=BEST_TRIGGER_THRESHOLD,
        strategy_name=BEST_STRATEGY
    )

    if len(baseline_scores_df) > 0:
        baseline_name = baseline_scores_df.iloc[0]["rare_tail_genre_name"]
        baseline_score = float(baseline_scores_df.iloc[0]["stage2_score"])
        baseline_hit = baseline_scores_df.iloc[0]["rare_tail_genre_id"] in
item["true_rare_ids"]
    else:
        baseline_name = None
        baseline_score = np.nan
        baseline_hit = False

    if len(best_scores_df) > 0:
        best_name = best_scores_df.iloc[0]["rare_tail_genre_name"]
        best_score = float(best_scores_df.iloc[0]["stage2_score"])
        best_hit = best_scores_df.iloc[0]["rare_tail_genre_id"] in
item["true_rare_ids"]
    else:
        best_name = None
        best_score = np.nan
        best_hit = False

    candidate_hit_any = any(gid in item["predicted_candidate_ids"] for gid in
item["true_candidate_ids"]) if len(item["true_candidate_ids"]) > 0 else False

    final_rows.append({
        "track_id": track_id,
        "title": item["title"],
        "true_candidate_label_names": item["true_candidate_names"],
        "predicted_candidate_label_names": item["predicted_candidate_names"],
        "candidate_hit_any": candidate_hit_any,
        "true_rare_tail_label_names": item["true_rare_names"],
        "baseline_stage2_suggestion": baseline_name,
        "baseline_stage2_score": baseline_score,
        "baseline_rare_tail_hit": baseline_hit,
    })

```

```
        "best_stage2_strategy": BEST_STRATEGY,  
        "best_stage2_suggestion": best_name,  
        "best_stage2_score": best_score,  
        "best_rare_tail_hit": best_hit  
    })  
  
final_targeted_test_results_df = pd.DataFrame(final_rows)  
  
print("Final targeted test results table:")  
display(final_targeted_test_results_df)
```

Final targeted test results table:

	track_id	title	true_candidate_label_names	predicted_candidate_label_names	car
0	16930	Open Session	[International, Spoken, Radio, Middle East]	[Avant-Garde, Rock, Experimental, Improv]	
1	30180	Conic Sections	[Novelty, Pop, Spoken, Experimental]	[Pop, Rock, Folk, Lo-Fi, Experimental]	
2	30181	Elliptically	[Novelty, Pop, Spoken, Experimental]	[International, Pop, Rock, Folk, Experimental]	
3	30182	Hyperboli	[Novelty, Pop, Spoken, Experimental]	[International, Blues, Pop, Rock, Folk, Experi...	
4	30183	Directrix	[Novelty, Pop, Spoken, Experimental]	[Pop, Rock, Punk, Lo-Fi, Experimental]	
5	30184	So It Goes	[Novelty, Pop, Spoken, Experimental]	[International, Blues, Pop, Rock, Folk, Experi...	
6	30185	Quitting Time	[Novelty, Pop, Spoken, Experimental]	[Pop, Rock, Folk, Experimental]	
7	30186	Roundabout	[Novelty, Pop, Spoken, Experimental]	[Pop, Rock, Electronic, Hip-Hop, Experimental, ...]	
8	30187	Inscribed Pythagorus	[Novelty, Pop, Spoken, Experimental]	[International, Pop, Rock, Electronic, Folk, H...	
9	30188	The Electricity Song	[Novelty, Pop, Spoken, Experimental]	[Avant-Garde, International, Pop, Rock, Folk, ...]	
10	30189	Unit Circle Trigonometry	[Novelty, Pop, Spoken, Experimental]	[Pop, Rock, Folk, Experimental]	
11	35853	Untitled	[International, Novelty, Rock, Punk, Experimen...	[Rock, Folk, Lo-Fi, Experimental]	
12	35854	Untitled	[International, Novelty, Rock, Punk, Experimen...	[Rock, Folk, Experimental, Instrumental]	
13	35857	Fanfare for the King of Tonga	[International, Novelty, Rock, Punk, Experimen...	[Avant-Garde, Rock, Experimental, Improv]	
14	35858	The King's Swim Fins	[International, Novelty, Rock, Punk, Experimen...	[Avant-Garde, Experimental, Improv]	
15	51013	The City	[Country, Rock, Ambient, Shoegaze, Country & W...	[Avant-Garde, Classical, Rock, Electronic, Sou...	
16	51014	Black-Bar Trombone	[Country, Rock, Ambient, Shoegaze, Country & W...	[Avant-Garde, Rock, Electronic, Hip-Hop, Noise...	
17	99351	Ille de Roman Olsun	[International, Folk, Middle East]	[International, Pop, Rock, Folk, Punk, Lo-Fi, ...]	
18	99352	Osman Aga	[International, Folk, Middle East]	[Rock, Punk, Experimental, Improv]	

	track_id	title	true_candidate_label_names	predicted_candidate_label_names	car
19	99353	Zeytinyagli	[International, Folk, Middle East]	[International, Pop, Rock, Folk, Punk, Experim...	
20	99354	Sevdam Olmasa / Muhabbet Bagi	[International, Folk, Middle East]	[Rock, Electronic, Hip-Hop, Experimental]	
21	99355	Kim Arar	[International, Folk, Middle East]	[Rock, Experimental]	
22	108156	Río Rivadavia	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Punk, Experimental]	
23	108157	Autoayuda	[International, Folk, Latin America, Europe, S...	[International, Pop, Rock, Folk, Punk, Experim...	
24	108158	Spring inside	[International, Folk, Latin America, Europe, S...	[Rock, Folk, Experimental, Singer-Songwriter]	
25	108159	Nocturno #2	[International, Folk, Latin America, Europe, S...	[Rock, Folk, Experimental, Singer-Songwriter]	
26	108160	Aloe	[International, Folk, Latin America, Europe, S...	[Rock, Electronic, Folk, Experimental, Singer-...	
27	108161	Pasará	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Experimental, Singer-Songwri...	
28	108162	Fantasia en G para un pueblo al sur del mapa	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Punk, Indie-Rock, Singer-Son...	
29	108163	Trovadora del Viento	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Punk, Experimental]	
30	108164	Por mí, que llueva	[International, Folk, Latin America, Europe, S...	[Country, Pop, Rock, Folk, Singer-Songwriter]	
31	108165	Hace frío (en NYC)	[International, Folk, Latin America, Europe, S...	[International, Pop, Rock, Folk, Punk, Experim...	
32	108166	Verde	[International, Folk, Latin America, Europe, S...	[International, Rock, Folk, Experimental, Sing...	
33	108167	Fruto prohibido	[International, Folk, Latin America, Europe, S...	[Pop, Rock, Folk, Experimental, Singer-Songwri...	
34	146485	Live on WFMU at Monty Hall/Transpacific Sound ...	[International, Indian]	[Avant-Garde, International, Rock, Electronic,...	

In [15]:

```
# =====  
# 15. BUILD BEST-STRATEGY LABEL-LEVEL SUMMARY
```

```
# =====

label_level_rows = []

for gid in fallback_rare_ids:
    gname = genre_name_map.get(gid, str(gid))

    true_mask = final_targeted_test_results_df["true_rare_tail_label_names"].apply(
        lambda x: gname in x if isinstance(x, list) else False
    )
    pred_mask =
final_targeted_test_results_df["best_stage2_suggestion"].fillna("").eq(gname)

    true_count = int(true_mask.sum())
    predicted_count = int(pred_mask.sum())
    hit_count = int((true_mask & pred_mask).sum())

    label_level_rows.append({
        "rare_tail_genre_id": gid,
        "rare_tail_genre_name": gname,
        "true_track_count": true_count,
        "predicted_track_count": predicted_count,
        "hit_count": hit_count,
        "label_recall": (hit_count / true_count) if true_count > 0 else np.nan
    })

best_label_level_df = pd.DataFrame(label_level_rows).sort_values(
    ["true_track_count", "hit_count", "rare_tail_genre_name"],
    ascending=[False, False, True]
).reset_index(drop=True)

print("Best-strategy rare-tail label-level summary:")
display(best_label_level_df)
```

Best-strategy rare-tail label-level summary:

	rare_tail_genre_id	rare_tail_genre_name	true_track_count	predicted_track_count	hit_count
0	1060	Tango	12	11	5
1	465	Musical Theater	10	0	0
2	1032	Turkish	5	5	3
3	176	Pacific	4	10	2
4	493	Western Swing	2	4	0
5	189	Talk Radio	1	1	1
6	173	N. Indian Traditional	1	0	0
7	374	Banter	0	3	0
8	377	Deep Funk	0	0	0
9	808	Salsa	0	0	0



In [16]:

```
# =====
# 16. BUILD FINAL SUMMARY TABLE
# =====

final_summary_rows = [
    {
        "Component": "Targeted validation rare-tail search",
        "Status": "Complete",
        "Details": f"Compared {len(strategy_list)} Stage-2 strategies over
{len(trigger_grid)} trigger thresholds."
    },
    {
        "Component": "Best Stage-2 strategy",
        "Status": "Selected",
        "Details": f"{BEST_STRATEGY} at anchor trigger threshold
{BEST_TRIGGER_THRESHOLD:.2f}."
    },
    {
        "Component": "Baseline Stage-2 test hit-any rate",
        "Status": "Measured",
        "Details": f"{baseline_test_summary['Rare-Tail Hit-Any Rate']:.4f}"
    },
    {
        "Component": "Best Stage-2 test hit-any rate",
        "Status": "Measured",
        "Details": f"{best_test_summary['Rare-Tail Hit-Any Rate']:.4f}"
    },
    {
        "Component": "Baseline Stage-2 test top-1 hit rate",
        "Status": "Measured",
        "Details": f"{baseline_test_summary['Rare-Tail Top-1 Hit Rate']:.4f}"
    },
    {

```

```

        "Component": "Best Stage-2 test top-1 hit rate",
        "Status": "Measured",
        "Details": f"{best_test_summary['Rare-Tail Top-1 Hit Rate']:.4f}"
    }
]

final_stage2_summary_df = pd.DataFrame(final_summary_rows)

print("Final Stage-2 summary:")
display(final_stage2_summary_df)

```

Final Stage-2 summary:

	Component	Status	Details
0	Targeted validation rare-tail search	Complete	Compared 6 Stage-2 strategies over 4 trigger t...
1	Best Stage-2 strategy	Selected	baseline_prior at anchor trigger threshold 0.05.
2	Baseline Stage-2 test hit-any rate	Measured	0.2857
3	Best Stage-2 test hit-any rate	Measured	0.3143
4	Baseline Stage-2 test top-1 hit rate	Measured	0.2857
5	Best Stage-2 test top-1 hit rate	Measured	0.3143

In [17]:

```

# =====
# 17. SAVE OUTPUTS
# =====

os.makedirs("../data/processed", exist_ok=True)

validation_search_df.to_csv(
    "../data/processed/full161_stage2_reranker_validation_search.csv",
    index=False
)

comparison_df.to_csv(
    "../data/processed/full161_stage2_reranker_test_comparison.csv",
    index=False
)

final_targeted_test_results_df.to_csv(
    "../data/processed/full161_stage2_reranker_targeted_test_results.csv",
    index=False
)

best_label_level_df.to_csv(
    "../data/processed/full161_stage2_reranker_label_level_summary.csv",
    index=False
)

final_stage2_summary_df.to_csv(

```

```

    "../data/processed/full161_stage2_reranker_final_summary.csv",
    index=False
)

with open("../data/processed/full161_stage2_reranker_best_config.txt", "w") as f:
    f.write(f"best_strategy={BEST_STRATEGY}\n")
    f.write(f"best_trigger_threshold={BEST_TRIGGER_THRESHOLD}\n")

if len(val_problem_files) > 0:
    pd.DataFrame(val_problem_files).to_csv(
        "../data/processed/full161_stage2_reranker_validation_problem_files.csv",
        index=False
    )

if len(test_problem_files) > 0:
    pd.DataFrame(test_problem_files).to_csv(
        "../data/processed/full161_stage2_reranker_test_problem_files.csv",
        index=False
    )

print("Saved Stage-2 rare-tail reranker outputs.")

```

Saved Stage-2 rare-tail reranker outputs.

In [18]:

```

# =====
# 18. INTERPRETATION NOTES
# =====

print("1. This notebook tests whether prototype and nearest-neighbor reranking can improve rare-tail routing.")
print("2. The Stage-1 candidate benchmark remains frozen and unchanged.")
print("3. The only thing being improved here is Stage-2 rare-tail selection.")
print("4. Validation is used to choose the best reranking rule before testing it on targeted rare-tail tracks.")
print("5. The outputs will show whether rare-tail reranking meaningfully improves over the baseline anchor-only router.")

```

1. This notebook tests whether prototype and nearest-neighbor reranking can improve rare-tail routing.
2. The Stage-1 candidate benchmark remains frozen and unchanged.
3. The only thing being improved here is Stage-2 rare-tail selection.
4. Validation is used to choose the best reranking rule before testing it on targeted rare-tail tracks.
5. The outputs will show whether rare-tail reranking meaningfully improves over the baseline anchor-only router.